FoundYou

Multi-Camera Person Re-Identification System

Developer Wiki & Architecture Reference

1. System Overview

FoundYou is a real-time multi-camera person re-identification (ReID) system. It ingests video streams from N simultaneous cameras, detects people with YOLO+ByteTrack, extracts appearance embeddings with a custom OSNet backbone, and maintains a unified gallery of global identities across all cameras.

The core problem it solves: when the same person moves from one camera's field of view to another, they should retain the same global ID rather than being assigned a new one. FoundYou achieves this through a three-step "resolve" pipeline executed on every new local tracklet.

> **Key Insight**
> Every detected person carries a local ID (assigned by ByteTrack per camera) and a global ID (assigned by FoundYou across all cameras). The system's job is to make those global IDs consistent and persistent across cameras and time.

1.1 High-Level Data Flow

Stage	Component	Description
`Frame capture`	camera.py	One CameraWorker thread per camera. Reads frames, runs YOLO+ByteTrack, crops detections.
`Feature extraction`	models.py	OSNet backbone extracts 512-dim L2-normalised embedding per crop.
`ReID resolve`	reid.py	GlobalReIDManager assigns or creates a global ID (gid) for each local tracklet.
`State storage`	tracks.py	GlobalTrack + DB store feature galleries, FAISS index, and lifecycle state.
`Rendering`	ui.py	Draws boxes, IDs, sidebar, and validation overlays.
`Benchmarking`	metrics.py + evaluate.py	Offline MOTA/IDF1/gallery quality metrics against labeled ground truth.

2. Module Reference

2.1 config.py — Configuration

Single source of truth for all thresholds and runtime constants. No project imports — safe to import from anywhere. All fields are class attributes on Config and can be overridden at startup via CLI args or Config.override_from_args(args).

Parameter	Default	Description
CROSS_CAM_DIST	0.40	Cosine distance threshold for cross-camera identity linking.
SAME_CAM_REENTRY_DIST	0.30	Tighter threshold for re-entry on the same camera (appearance barely changes).
REACQ_DIST	0.40	Threshold for reacquiring an archived track.
REACQ_MIN_SIM	0.60	FAISS inner-product pre-filter floor (1 − REACQ_DIST).
MIN_PROBES_TO_MATCH	3	Gallery must have this many feature vectors before it can claim new tracklets.
MERGE_DIST	0.35	Post-hoc merge: two active tracks below this distance are collapsed.
MERGE_INTERVAL	2.0s	How often to run the active-track merge pass.
CAM_FEAT_BUF	24	Maximum feature vectors stored per camera per track.
INACTIVE_TTL	15.0s	Seconds before an unobserved active track is archived.
EXPIRE_LOCAL_TTL	4.0s	Seconds before a local tracklet slot is released.
FACE_DETECTION	True	Master switch for face-detection pipeline.
FACE_VETO_DIST	0.55	Cosine distance above which faces veto a body-ReID match.
FACE_WEIGHT	0.30	Weight of face distance in combined matching score.
FLIP_CAMERAS	[]	Camera IDs whose frames are horizontally flipped before processing.
DEVICE	auto	Torch device: mps > cuda > cpu.

2.2 models.py — Feature Extractor

Implements the OSNet appearance encoder. On startup it tries to load a pretrained OSNet via torchreid; if not installed it falls back to a custom OSNet implementation and downloads weights from Google Drive.

- Architecture: OSNet-x1.0 with four OSBlock layers and omni-scale feature learning.
- Input: BGR crop (any size) — resized to 256×128, ImageNet-normalised.
- Output: 512-dim float32 unit-norm embedding.
- TTA: forward pass averaged with horizontally-flipped pass (test-time augmentation).
- Batch processing: all crops in a frame processed together; falls back to per-crop on OOM.

2.3 camera.py — Camera Worker

CameraWorker is a daemon Thread, one per camera. It runs in a tight loop reading frames and pushing results to a shared output queue.

- YOLO tracking: ultralytics YOLO.track() with persist=True and ByteTrack tracker.
- Filtering: boxes smaller than 20×40 px are discarded.
- Camera flip: if cam_id is in Config.FLIP_CAMERAS the frame is horizontally flipped before any processing.
- Face detection: FaceDetector.detect() is called per crop and stored in meta['face'].
- Dead sentinel: if YOLO fails to load, a DEAD_SENTINEL is pushed to the queue and the worker exits.

Thread Safety

FaceDetector uses cv2.dnn which is not thread-safe. One FaceDetector instance is created per CameraWorker thread.

2.4 face.py — Face Detection

Optional pre-ReID face gate. Runs per detection crop on the upper FACE_SEARCH_FRACTION of the bounding box. Results are stored as FaceResult namedtuples on each detection dict.

- FaceResult fields: found, embedding (optional), quality (0–1 sharpness×size×aspect score), conf.
- face_gate(face_a, face_b): returns False (veto) if both have usable embeddings and their cosine distance exceeds FACE_VETO_DIST.
- face_distance(face_a, face_b): cosine distance between two face embeddings; returns 2.0 if either is unusable.
- Combined score: combined = (1−FACE_WEIGHT)×body_dist + FACE_WEIGHT×face_dist, only when both sides have usable faces.

2.5 features.py — Feature Utilities

Pure-numpy distance helpers. No circular imports — nothing here imports tracks or reid.

Function	Signature	Description
`_norm(v)`	ndarray | None	Returns unit-norm copy of v, or None on bad input.
`_dist(a, b)`	float [0, 2]	Cosine distance. 0 = identical, 1 = orthogonal, 2 = None input.
`_robust_dist_to_gallery(feat, gt)`	float [0, 2]	FP-1 fix: per-camera median distance averaged with merged-mean distance.

Why Robust Distance?

_robust_dist_to_gallery() takes the median of distances to each stored frame per camera (instead of min). This kills the effect of one blurred/noisy frame that happens to be accidentally close to a query — which is exactly what a naive min() was vulnerable to.

2.6 tracks.py — Track Data Model & Persistence

GlobalTrack

One object per identity. Holds the feature gallery, lifecycle timestamps, and local tracklet occupancy state.

Field	Type	Description
`gid`	int	Globally unique identity number (auto-incremented).
`_buf`	dict[cam → list[ndarray]]	Per-camera rolling buffer of recent feature vectors (max CAM_FEAT_BUF).
`feature`	ndarray | None	Mean-of-camera-means merged gallery embedding (rebuilt on every observe()).
`locals`	dict[(cam, lid) → ts]	Active local tracklets currently pointing to this track.
`cams_seen`	set[int]	All cameras this person has ever been observed on.
`archived`	bool	True once the track has been inactive for INACTIVE_TTL seconds.
`det_count`	int	Total detections ever associated with this track.
`color`	tuple	Deterministic BGR display colour seeded from gid.

Key methods:

- observe(cam, lid, feat, ts): append feature to buffer, rebuild merged gallery, update locals.
- is_matchable(): returns True only if total_probes() >= MIN_PROBES_TO_MATCH (FP-3 fix).
- cam_is_occupied(cam, ts): True if a live local tracklet on cam already maps here.

DB

FAISS flat inner-product index combined with a pickle-serialised list of GlobalTrack objects. The FAISS index is used only for coarse archived-track retrieval (step B); robust verification follows.

- upsert(gt): add new or update existing track. Does NOT update FAISS immediately.
- search_archived(feat, k=20): FAISS search over archived tracks, returns (gid, sim, gt) triples filtered by REACQ_MIN_SIM.
- rebuild_index(): full FAISS rebuild — called periodically and on startup.
- cleanup(ts): delete tracks archived > CLEANUP_AFTER seconds ago.

2.7 reid.py — Global ReID Manager

The heart of the system. GlobalReIDManager owns the DB, the active track dict, and the local-to-global mapping (l2g). It exposes update() called every frame by main.py.

The Resolve Pipeline

Every new local tracklet (cam, local_id) that is not already in l2g passes through _resolve_with_face():

Step	Name	Logic
A	Match active track	Iterate active tracks. Skip if camera occupied or gallery not matchable. Compute _robust_dist_to_gallery(). Apply SAME_CAM_REENTRY_DIST or CROSS_CAM_DIST threshold. Apply face veto + combined score. Pick single best match.
B	Reacquire archived	FAISS coarse filter over archived tracks. Robust distance verify. Apply REACQ_DIST threshold. Face veto. Unarchive the track.
C	Create new track	Allocate a fresh GlobalTrack. Always fires when A and B both fail.

Post-Hoc Merge Pass

Every MERGE_INTERVAL seconds, _merge_active_tracks() compares all pairs of matchable active tracks. If their robust distance < MERGE_DIST AND they are not simultaneously live on the same camera, the younger/smaller track is folded into the older/larger one.

This corrects the common early-fragmentation failure: two global IDs are assigned to the same person before either gallery reached MIN_PROBES_TO_MATCH.

Four False-Positive Fixes

FP-1: Robust distance replaces naive min(). FP-2: Per-scenario thresholds (SAME_CAM vs CROSS_CAM). FP-3: MIN_PROBES_TO_MATCH gate prevents unripe galleries from stealing IDs. FP-4: Tighter REACQ_DIST for archive reacquisition.

2.8 id_validator.py — Runtime ID Validator

Periodic consistency checker that detects ID problems without mutating state. The merge pass fixes; the validator surfaces what the merge pass missed.

Issue	Trigger	Root cause
SWAP	Two tracks on different cameras have robust distance < ID_SWAP_DIST	Flipped camera, low probe count at match time, or threshold too loose.
DUPLICATE	Two tracks both live on same camera have distance < MERGE_DIST	ByteTrack split one person; merge pass not yet fired.
UNSTABLE	Intra-gallery mean pairwise distance > 0.30	Buffer accumulated frames from more than one person.

Issues gain a streak count across consecutive checks. streak >= 3 = persistent; severity = HIGH at streak >= 5. Persistent issues are logged at WARNING level.

2.9 main.py — Entry Point

Wires all modules together. Owns the main loop, camera discovery, and graceful shutdown.

- Camera discovery: probes indices 0..MAX_CAMERAS-1 with all platform backends to find physical and virtual cameras.
- Threading model: one CameraWorker thread per camera pushes to a shared Queue. Main thread drains the queue, calls mgr.update(), renders, and handles keyboard input.
- Session recording: optional --record flag wraps mgr with SessionRecorder for offline benchmarking.
- Keyboard controls: Q=quit, D=toggle debug, S=print stats, V=print validation report, X=screenshot.

2.10 ui.py — Rendering

Pure OpenCV/numpy. Stateless functions called from the main loop.

- draw_detections(frame, dets, mgr, ts, val_issues): overlays bounding boxes, global ID labels, and pulsing-red swap warnings.
- _wrap_camera(frame, cid, ...): adds padding and an ID-strip with coloured pill badges showing each visible global ID.
- assemble_video(frames_ready, ...): tiles wrapped camera frames (1/2/3/4-camera layouts).
- draw_sidebar(sb, mgr, ts, fps, val_issues): renders the 420 px right panel with system stats, thresholds, counters, validation summary, and active ID list.

3. Benchmarking Pipeline

3.1 metrics.py — All Metric Implementations

Pure-numpy, no live-system dependency. Accepts Detection and TrackingResult dataclasses.

Shared Types

Type	Fields	Notes
`Detection`	frame, cam, true_id, pred_id, bbox	true_id=0 means unknown/ignored. pred_id=0 means GT (no prediction).
`TrackingResult`	step_a, step_b, step_c, merges, total_time	Accumulated from live run or loaded from events.csv.

ReID Ranking Metrics

- compute_distmat(q, g): pairwise cosine distance (Q×G matrix).
- compute_cmc(distmat, q_pids, g_pids, q_camids, g_camids): Market-1501 CMC with junk removal.
- rank_k_accuracy(cmc, k): Rank-k accuracy from CMC curve.
- mean_ap(distmat, ...): mean Average Precision over all queries.

Multi-Object Tracking Metrics

Metric	Formula	Interpretation
`MOTA`	1 − (FN + FP + IDS) / GT	Primary tracking accuracy. 1.0 = perfect, can be negative.
`MOTP`	sum(IoU of matches) / N_matches	Localisation precision. 1.0 = perfect box overlap.
`IDF1`	2·IDTP / (2·IDTP + IDFP + IDFN)	Identity F1. Primary ReID quality metric.
`IDP`	IDTP / (IDTP + IDFP)	ID precision: fraction of predictions with correct ID.
`IDR`	IDTP / (IDTP + IDFN)	ID recall: fraction of GT with a correct prediction.
`IDS`	count	Raw ID switches: GT person changes predicted ID between frames.
`Frags`	count	Track interruptions: GT track goes present→absent→present.

Spatial Matching

All per-frame metrics use greedy IoU matching (highest IoU pair first). iou_thr=0.5 by default. Adjust with --iou-thr on run_benchmark.py.

Gallery Quality Metrics

- gallery_dist_stats(tracks): intra-gallery (consistency) and inter-gallery (separation) cosine distances. separation_ratio = inter_mean / intra_mean.
- probe_convergence(track): simulates adding probes one at a time; measures how quickly the gallery mean stabilises.

Pipeline Metrics

- resolve_step_breakdown(result): fractions of resolves handled by Step A / B / C.
- merge_effectiveness(result, gt_frag): merges_fired vs fragmentations actually present in GT.
- throughput(result, total_dets): detections/sec and resolves/sec.

3.2 data.py — Benchmark Data Utilities

Loaders for annotation formats and synthetic data generation.

Function	Input	Description
`load_csv(path)`	CSV: frame,cam,pid,x1,y1,w,h	Load MOT-style multi-camera annotation.
`load_mot_txt(path, cam_id)`	MOT17/20 .txt	Load single-camera MOT ground truth.
`load_mot_sequence(seq_dir)`	cam0/gt/gt.txt …	Load all cameras in a sequence directory.
`generate_synthetic_sequence(...)`	Parameters	Generate GT + predicted sequence with injected errors for unit testing.
`detections_from_log(log_path)`	FoundYou debug log	Parse log file to extract predicted detections.
`detections_from_snapshot(...)`	Live system state	Convert one frame's raw detection dicts to Detection objects.

generate_synthetic_sequence() is the key tool for unit testing metrics without video. It creates N people walking between M cameras with configurable ID switches and fragmentations injected at random frames.

3.3 ablation.py — Ablation Runner

Evaluate multiple Config variants side-by-side and compare metric outcomes. The core tool for ablation tables.

- run_ablation(gt, sequences, variants, ...): evaluate each variant, return list of result dicts.
- print_table(results): fixed-width ASCII table to stdout.
- save_csv(results, path): save results dict list to CSV.
- delta_table(baseline, variants): compute per-metric delta vs baseline row.

Caller Responsibility

run_ablation() documents variant Config overrides but does not re-run FoundYou with those settings. The caller must supply one predicted detection sequence per variant (obtained by actually running the system with each Config).

3.4 recorder.py — Session Recorder

Wraps GlobalReIDManager at runtime to record all detections and pipeline events. Monkey-patches mgr._resolve and mgr._merge_active_tracks to capture step-level decisions without modifying reid.py.

Output file	Format	Contents
`detections.csv`	frame,cam,local_id,global_id,x1,y1,x2,y2,ts	Every predicted detection per frame.
`events.csv`	ts,event_type,gid,cam,local_id,detail	step_a_cross / step_a_reentry / step_b_reacq / step_c_new / merge.
`config_snapshot.json`	JSON	Config field values at recording time for reproducibility.
`metadata.json`	JSON	Camera count, frame counts, total duration, step counts.

Integration: call recorder.record_frame(cid, frame_counter[cid], dets) after each mgr.update(). Call recorder.save() in the finally block. Enable with --record flag on main.py.

3.5 labeler.py — Ground Truth Labeler

OpenCV-based interactive tool to assign ground-truth person IDs to a recorded session. Opens each camera's recorded frames (or a video file), lets you click bounding boxes and type the real person ID.

Control	Action
`Click box`	Select it (turns yellow)
`Type digits`	Set person ID for selected box
`Enter`	Confirm; propagates to all frames where this global ID appears
`N / →`	Next frame
`P / ←`	Previous frame
`S`	Save gt_cam{N}.csv and quit
`Q`	Quit without saving

Output: gt_cam{N}.csv per camera with columns frame,cam,true_id,global_id,x1,y1,x2,y2. The load_labeled_session() function merges all camera labels into unified GT and PR Detection lists ready for metrics.

3.6 evaluate.py — Session Evaluator

High-level CLI that runs the full metric suite against a recorded+labeled session directory.

```
python -m evaluate --session recordings/session_20240101_120000
python -m evaluate --sessions s1 s2 --names baseline tight --ablation --out results.csv
```

3.7 run_benchmark.py — Benchmark Runner

End-to-end benchmark. Can run against real annotation CSVs, synthetic data, or an ablation demo.

```
python -m run_benchmark --synthetic
python -m run_benchmark --synthetic --ablation --out results.csv
python -m run_benchmark --gt path/gt.csv --pred path/pred.csv --iou-thr 0.5
```

3.8 tests.py — Unit Tests

Self-contained metric unit tests using only synthetic data. No cameras, no GPU required.

```
python -m tests
```

Coverage: perfect/all-FP/ID-switch MOTA, perfect/no-match MOTP, perfect/wrong-ID IDF1, fragmentation injection, CMC/mAP perfect retrieval, gallery stats keys, step fraction arithmetic, merge effectiveness, throughput, ablation smoke test.

4. End-to-End Data Flow

4.1 Live System Flow

The following sequence describes one pass through the main loop:

- **CameraWorker threads push (cam_id, frame, metas) into result_q. Each meta dict contains local_id, bbox, feat, face, cx, cy.**1.
- **Main thread drains result_q; stores latest frame and detection list per camera.**2.
- **mgr.update(all_dets, ts) is called with a snapshot of all cameras' current detections.**3.
- **Inside update(): expire stale l2g entries → group by (cam, local_id) → for each group either observe() on existing track or call _resolve_with_face().**4.
- **Archive idle tracks; run merge pass if MERGE_INTERVAL elapsed; rebuild FAISS if dirty.**5.
- **IDValidator.check(ts) scans for SWAP/DUPLICATE/UNSTABLE issues.**6.
- **draw_detections() and draw_sidebar() render the frame; cv2.imshow() displays.**7.

4.2 Offline Benchmarking Flow

To benchmark a live session:

- 1. Record: run main.py --record. Saves detections.csv, events.csv, config_snapshot.json.
- 2. Label: run labeler.py --session <dir> --cam N. Repeat for each camera. Saves gt_cam{N}.csv.
- 3. Evaluate: run evaluate.py --session <dir>. Loads GT + PR, computes all metrics, prints report.
- 4. Ablation: collect multiple session dirs (one per Config variant), run evaluate.py --sessions ... --ablation.

5. Identity Lifecycle

5.1 States

State	Condition	l2g / active entries
`Nascent`	track.total_probes() < MIN_PROBES_TO_MATCH	In active; can receive observations but cannot claim new tracklets.
`Active`	In mgr.active; last_seen within INACTIVE_TTL	One or more l2g entries pointing to this gid.
`Archived`	Not seen for INACTIVE_TTL seconds	Removed from mgr.active; stored in DB; searchable via FAISS.
`Cleaned`	Archived > CLEANUP_AFTER seconds ago	Deleted from DB and FAISS index.

5.2 Local Tracklet Expiry

A local tracklet (cam, local_id) is removed from gt.locals and l2g after EXPIRE_LOCAL_TTL seconds without an observation. This frees the camera slot so a different person can be linked to the track from that camera.

5.3 Gallery Rebuild

Every call to gt.observe() appends the incoming feature to the per-camera rolling buffer (_buf[cam], max CAM_FEAT_BUF) and calls _rebuild(). _rebuild() computes the mean-of-camera-means as the merged gallery feature vector stored in gt.feature.

6. Configuration & Tuning Guide

6.1 Diagnosis First

Before changing thresholds, check the sidebar stats and run IDValidator. The step fraction breakdown is the most useful first signal:

Symptom	Likely cause	Suggested fix
`High Step-C fraction`	Too many new tracks; CROSS_CAM_DIST too tight or MIN_PROBES too high	Loosen CROSS_CAM_DIST; reduce MIN_PROBES_TO_MATCH.
`High ID switches`	CROSS_CAM_DIST too loose; camera flip needed	Tighten CROSS_CAM_DIST; add camera to FLIP_CAMERAS.
`Persistent SWAP issues`	Cameras have very different viewpoints/lighting	Lower ID_SWAP_DIST; increase FACE_WEIGHT if faces usable.
`Persistent DUPLICATE issues`	MERGE_DIST too tight	Increase MERGE_DIST slightly.
`People not reacquired after leaving`	REACQ_DIST too tight or INACTIVE_TTL too short	Loosen REACQ_DIST; increase INACTIVE_TTL.

6.2 Camera Flip

If one camera is physically mirrored (mounted upside-down or behind a mirror), embeddings from that camera will be systematically different from all others, causing persistent SWAP issues. Add the camera ID to FLIP_CAMERAS:

```
python main.py --flip-cameras 1
```

The flip happens before YOLO, ByteTrack, AND the ReID extractor, so embeddings become directly comparable across cameras.

6.3 Face Module

Face detection is on by default. Disable globally with FACE_DETECTION=False. When enabled:

- FACE_EMBEDDING=True: extract face embeddings (requires insightface or similar); use for VETO + blending.
- FACE_EMBEDDING=False: detect face presence only; use as a quality gate (has_face) without embedding-based veto.
- FACE_WEIGHT=0.0: ignore face entirely in the combined score even if embedding is available.

7. Quick Start

7.1 Install

```
pip install ultralytics faiss-cpu numpy opencv-python torch torchvision
pip install torchreid  # optional but strongly recommended
pip install insightface  # optional for face embeddings
```

7.2 Run

```
python main.py
python main.py --flip-cameras 1 --cross-cam-dist 0.35 --debug
python main.py --record --record-dir recordings/
```

7.3 Benchmark (Synthetic)

```
python -m run_benchmark --synthetic
python -m run_benchmark --synthetic --ablation --out ablation.csv
python -m tests  # unit tests
```

7.4 Benchmark (Real Session)

```
# 1. Record
python main.py --record
# 2. Label each camera
python -m labeler --session recordings/session_X --cam 0
python -m labeler --session recordings/session_X --cam 1
# 3. Evaluate
python -m evaluate --session recordings/session_X
```

8. Glossary

Term	Definition
`local_id (lid)`	Per-camera tracklet ID assigned by ByteTrack. Resets across cameras.
`global_id (gid)`	System-wide identity number assigned by FoundYou. Persistent across cameras.
`l2g`	Dict mapping (cam, local_id) → gid. The core routing table.
`probe`	A single feature vector observation stored in a track's gallery buffer.
`matchable`	A track with total_probes() >= MIN_PROBES_TO_MATCH.
`archived`	A track that has been inactive for > INACTIVE_TTL. Searchable via FAISS.
`robust distance`	Per-camera median distance averaged with merged-mean distance.
`MOTA`	Multi-Object Tracking Accuracy: 1 − (FN+FP+IDS)/GT.
`IDF1`	Identification F1: primary ReID quality metric. Penalises persistent ID confusion.
`MOTP`	Multi-Object Tracking Precision: average IoU of matched pairs.
`step_a / B / C`	Active match / archived reacquisition / new track creation.
`separation_ratio`	inter_mean / intra_mean. Higher = identities are more distinct in embedding space.
`fragmentation`	A GT track going present → absent → present (each interruption counted once).